

Test- en Validatieoverzicht voor Klantreview

Datum: 2026-05-06

Dit document legt in gewone taal uit hoe de Apex Rainier Planning Tool is getest. Het doel is dat zowel technische als niet-technische reviewers kunnen zien:

- welke onderdelen automatisch getest zijn
- welke onderdelen handmatig door gebruikersflows zijn gevalideerd
- wat het gemeten percentage automatische code-dekking is
- welke beperkingen nog openstaan

Korte conclusie

De applicatie is breed getest op de belangrijkste klantflows: workbook laden, berekenen, planning aanpassen, machines/capaciteit controleren, waarden bekijken, sessies beheren, scenario's gebruiken en exporteren.

Op basis van het actuele volledige non-browser coverage-rapport is de branch-enabled coverage-score nu **81%**. De statement coverage is **83,9%** van de uitvoerbare Python-regels. Dat betekent concreet:

Scope	Uitvoerbare regels	Automatisch geraakt	Percentage
Totaal <code>modules/</code> + <code>ui/</code>	5.577	4.678	83,9% statement / 81% branch-enabled
Rekenmodules <code>modules/</code>	2.990	2.331	78,0% statement
Web/UI-laag <code>ui/</code>	2.587	2.347	90,7% statement

Aanvullend:

- branch coverage staat aan voor rapportage
- de fixture-free CI-slice haalt **62%** coverage
- de browsertest-suite telt **19 Playwright-validaties**

Belangrijk om goed te interpretern:

- Dit percentage is **code coverage**: het meet hoeveel uitvoerbare Python-code door automatische tests is uitgevoerd.
- Het is **geen garantie dat 81% van alle mogelijke business-situaties is bewezen**. Een regel kan geraakt zijn zonder iedere randvoorwaarde te testen.
- De kritieke klantflows worden extra afgedekt door browsertests en een human-verification workbook. Daardoor is de praktische validatie sterker dan alleen het coverage-percentage suggereert.
- Visuele juistheid van charts en heatmaps is deels automatisch en deels handmatig gevalideerd.

Wat betekent “automatisch getest”?

Automatisch getest betekent dat een test zonder menselijke handelingen de code uitvoert en controleert of de uitkomst klopt. In dit project gebeurt dat op meerdere niveaus:

1. Unit tests controleren losse rekenmodules.
2. Integratietests controleren cascades, sessies, resets en replay.
3. API-tests controleren Flask endpoints en response-vormen.
4. Browsertests starten de echte webapp en klikken door gebruikersflows.
5. Golden-fixture tests vergelijken belangrijke berekeningen met een beheerde referentie-uitkomst.
6. Pipeline- en regressietests controleren dat belangrijke berekeningen en datastromen stabiel blijven.

Samen dekken deze tests zowel de rekenkern als veel van het gedrag dat een gebruiker in de browser ziet.

Wat is handmatig gevalideerd?

Niet alles is zinvol volledig automatisch te bewijzen. Charts, heatmapkleuren, visuele verversing en “ziet de gebruiker het juiste effect op de juiste plek?” zijn daarom ook met een human-verification workbook gecontroleerd.

Handmatige validatie betekent hier niet “losse indruk”; het gaat om gestructureerde teststappen met PASS-markeringen en screenshot/evidence sheets.

Gebruikte testdata

Golden fixture workbook

De automatische validatie gebruikt een beheerde golden fixture workbook. Dit is een representatieve planning-workbook met realistische volumes, BOM-structuur, routing, machinecapaciteit, veiligheidsvoorraad, purchase actuals, waarderingsparameters en exportdata.

Deze data is gebruikt voor:

- volledige engine smoke test via `python main.py --test`
- browser/end-to-end tests
- validatie van planning, capaciteit, values, inventory, sessies en exports
- vergelijking van belangrijke rekenuitkomsten met een bekende referentie-uitkomst

Human-verification workbook

Gebruikte workbook:

HUMAN_VERIFICATION_10D0F.xlsx

De workbook bevat 9 sheets:

Sheet	Doel
COVER	Instructies en sign-off flow voor testers
TEST A+B	Parallele test: Machines tab + Planning tab
Test A SS	Screenshot/evidence sheet voor TEST A+B
TEST C+D	Parallele test: charts/heatmap + edits
test CD SS	Screenshot/evidence sheet voor TEST C+D
TEST E	Solo full-workflow test
test E SS	Screenshot/evidence sheet voor TEST E
TEST F	Extra validatie voor nieuwe editable lines in de Machines tab
test F SS	Screenshot/evidence sheet voor TEST F

Aanvullende handmatige sign-off documenten:

Document	Doel
HUMAN_VERIFICATION_TESTS_H_I.md	Solo validatie voor nieuwe editable lines, cascade/reset/restart/sessie-isolatie, charts en export-integriteit
HUMAN_VERIFICATION_TESTS_H_I.pdf	Reviewbare PDF-versie van dezelfde H/I-testset

Status in workbook:

Testgroep	PASS-markeringen	Wat wordt vooral gecontroleerd
TEST A+B	18 ✓	Gelijktijdige machine- en planning-edits, capaciteit, dashboard
TEST C+D	18 ✓	Charts, heatmap thresholds, OEE/demand/availability effecten
TEST E	9 ✓	End-to-end workflow: planning edit, machines, dashboard, undo/redo/export
TEST F	✓ PASS	Nieuwe editable machine-lines, heatmap, charts, values en sessiegedrag
TEST H	20 ✓	L4/L7/L9/L11/L12 edits, cascade-matrix, reset, restart en sessie-isolatie
TEST I	20 ✓	Inventory quality charts, starting-stock visualisatie, data labels en Excel PNG-export

Deze tests zijn belangrijk omdat ze meerdere tabs en state-lagen in dezelfde sessie combineren. Daarmee vinden ze problemen die losse unit tests minder snel zien, bijvoorbeeld stale charts, verkeerd ververste heatmaps of edits die op een andere tab niet zichtbaar worden.

Tests H en I zijn toegevoegd als extra solo-validaties bovenop Tests A-G. Test H richt zich specifiek op de nieuwe editable lines en controleert dat live edits, reset, restart en replay hetzelfde sessiegebonden resultaat geven. Test I richt zich op chart-rendering en export-integriteit na de overstap naar ingebedde matplotlib PNG-grafieken, inclusief starting-stock visualisatie en data labels.

Automatische testlagen

1. Rekenmodule-tests

Voorbeelden:

- `tests/test_forecast_engine.py`
- `tests/test_bom_engine.py`
- `tests/test_inventory_engine.py`
- `tests/test_capacity_engine.py`
- `tests/test_value_planning_engine.py`
- `tests/test_inventory_quality_engine.py`
- `tests/test_mom_comparison_engine.py`

Deze tests controleren de losse rekenstappen:

- forecast mapping naar planningperiodes
- BOM parent/child dependent demand
- inventory, target stock, production plan en purchase receipt
- capacity utilization, available capacity, utilization rate en FTE
- value planning en consolidatie
- inventory quality en month-over-month vergelijking

Waarom dit belangrijk is:

De rekenmodules zijn de kern van de applicatie. Als hier een formule, datastroom of periodeverwerking verandert, kunnen planningtabellen, dashboards en exports direct andere cijfers tonen.

2. Edit- en cascade-tests

Voorbeelden:

- `tests/test_volume_change.py`
- `tests/test_edit_l4_starting_stock.py`
- `tests/test_edit_capacity_overrides.py`
- `tests/test_capacity_engine_overrides.py`
- `tests/test_capacity_overrides_persistence.py`

Deze tests controleren:

- Line 01 demand forecast edits
- Line 04 starting stock edit op de speciale startcel
- Line 07 capacity utilization override
- Line 09 available capacity override
- Line 11 shift availability override
- Line 12 FTE requirements override
- Line 10 blijft afgeleid en is niet direct editable
- live edit cascade versus rebuild/replay gedrag
- persistence van inventory- en capacity-overrides

Waarom dit belangrijk is:

Een gebruiker past de planning interactief aan. Na zo'n edit moeten downstream volumes, capaciteit, FTE, values, dashboarddata en exports dezelfde nieuwe engine-state gebruiken. De belangrijkste invariant is:

```
live edit sequence == clean rebuild + replay pending_edits
```

Dat betekent: als de applicatie opnieuw start en opgeslagen edits opnieuw afspeelt, moet dezelfde uitkomst ontstaan als tijdens de live gebruikerssessie.

3. State-, session- en persistence-tests

Voorbeelden:

- `tests/test_state_model.py`
- `tests/test_state_snapshot.py`
- `tests/test_session_store.py`
- `tests/test_engine_rebuild.py`
- `tests/test_replay.py`
- `tests/test_pending_edits.py`

Deze tests controleren:

- reset baselines
- pending edits
- replay na rebuild
- session metadata persistence
- value aux overrides
- machine overrides
- restore/snapshot van engine state

Waarom dit belangrijk is:

De webapp ondersteunt meerdere planning sessions. Correcte isolatie is essentieel: een edit in sessie A mag sessie B niet besmetten, en een restart mag edits/config niet verliezen.

4. Route/API-tests

Voorbeelden:

- `tests/test_routes_workflow.py`
- `tests/test_routes_sessions.py`
- `tests/test_routes_edits.py`
- `tests/test_routes_machines.py`
- `tests/test_routes_config.py`
- `tests/test_routes_read.py`
- `tests/test_routes_exports.py`
- `tests/test_routes_scenarios.py`
- `tests/test_routes_pap.py`

Deze tests controleren de Flask API-contracten:

- upload en calculate flow
- sessions list/switch/delete/rename/snapshot
- edit, undo, redo, reset, import/export edits
- machines update/reset/undo/redo
- config settings en folder settings
- dashboard/capacity/inventory/value read endpoints
- planning export, DB export en MoM
- scenario save/load/compare/export
- PAP als expliciete override route

Waarom dit belangrijk is:

De browser praat met deze endpoints. Route tests bewaken statuscodes, response-vormen en server-side side effects zonder dat een tester alles handmatig hoeft te klikken.

5. Browser/end-to-end tests

Voorbeelden:

- `tests/browser/test_load.py`
- `tests/browser/test_edits.py`
- `tests/browser/test_machines.py`
- `tests/browser/test_sessions.py`
- `tests/browser/test_charts.py`

Deze tests starten de echte webapp met Playwright en representatieve testdata. Ze controleren:

- pagina laadt zonder JavaScript errors
- planning table rendert met de verwachte periodes
- cell edit past de waarde aan en markeert pending edits
- undo werkt vanuit de UI
- machine edits verversen tabellen zonder tab switch
- machine undo/redo en save guards
- session list/switch/delete/rename
- saved instance heropent met pending edit
- dashboard-, values- en machine-charts renderen als nonblank canvassen

Waarom dit belangrijk is:

Dit benadert de echte gebruikersflow het meest: browser, Flask server, upload, calculate, DOM rendering, API calls en UI state werken samen.

6. Golden-fixture tests

De golden-fixture tests gebruiken een vaste referentie-workbook en controleren dat de belangrijkste rekenuitkomsten stabiel blijven. Dit voorkomt dat een wijziging in code ongemerkt andere planningwaarden oplevert.

Deze tests controleren onder andere:

- aanwezigheid van verwachte line types
- materiaalsets per line type
- numerieke planningwaarden binnen afgesproken toleranties
- stabiliteit van de demand-to-capacity-to-values pipeline

Waarom dit belangrijk is:

Unit tests bewijzen losse stukjes logica. De golden-fixture tests kijken naar het gecombineerde resultaat van de volledige planningketen. Daardoor zijn ze een sterke regressiecontrole voor klantrelevante berekeningen.

7. Cycle manager en MoM tests

Voorbeelden:

- `tests/test_cycle_manager.py`
- `tests/test_mom_comparison_engine.py`

Deze tests controleren:

- cyclus folder structuur en bestandsbeheer
- maandelijks naast elkaar vergelijken (Month-over-Month / MoM)
- MoM berekeningen: delta's, variances en trendanalyse
- export van MoM-rapporten

Waarom dit belangrijk is:

MoM-analyse is kritiek voor client-reporting. Een cycle manager moet vastgestelde périodes en hun onderliggende data correct ordenen en beheren. Fouten hier beïnvloeden directe klantuitvoeringen.

8. Chart renderer tests

Voorbeelden:

- `tests/test_chart_renderer.py`

Deze tests controleren:

- PNG-uitvoer van planning-, capaciteits- en valuecharts
- chart data labels en legendas
- heatmap rendering en thresholds
- visuele output zonder corruptie of missing data

Waarom dit belangrijk is:

Charts worden in dashboards en Excel-exports ingebed. Renderer-fouten leiden tot lege of beschadigde visuele output die de consultant niet kan gebruiken.

9. Data loader error handling tests

Voorbeelden:

- `tests/test_data_loader_errors.py`

Deze tests controleren:

- foutafhandeling voor ongeldige Excel-werkboeken
- fouten in SAP-data formaten (MS_RECONC)
- ontbrekende of beschadigde kolommen
- waarschuwingen en foutmeldingen naar gebruiker

Waarom dit belangrijk is:

Uploads van klanten kunnen onvolledig of misvormd zijn. Goede foutmeldingen helpen consultants snel te diagnosticeren en te herstellen in plaats van stille fouten.

Uitgevoerde verificatiecommando's

Compile check:

```
python -m py_compile tests/test_routes_workflow.py tests/test_routes_scenarios.py tests/test_routes_sessions.py tests/test_routes_config.py
```

Engine smoke:

```
python main.py --test
```

Resultaat:

```
Passed  
Line types generated: 15
```

Gerichte editable-line regressies:

```
pytest -v tests/test_capacity_engine_overrides.py tests/test_edit_capacity_overrides.py
```

Resultaat:

```
11 passed
```

Browser tests:

```
pytest -v tests/browser
```

Resultaat:

```
19 passed
```

Opmerking:

De laatste browser run meldde geen JavaScript console errors. De tests dekken nu naast DOM- en editgedrag ook nonblank canvas-rendering voor dashboard-, values- en machinecharts.

Full non-browser coverage:

```
pytest --ignore=tests/browser --cov=ui --cov=modules --cov-report=term --cov-report=html
```

Resultaat:

```
490 passed, 1 skipped, 1 warning  
TOTAL coverage: 81%
```

Fixture-free CI coverage:

```
pytest -m no_fixture --cov=ui --cov=modules
```

Resultaat:

```
434 passed, 1 skipped, 75 deselected  
TOTAL coverage: 62%
```

Dekking van charts en visuele correctheid

Charts zijn op twee manieren geraakt:

1. Automatisch:

- browser tests bewaken dat de app laadt en echte UI-acties kan uitvoeren
- Playwright canvas checks bewaken dat dashboard-, values- en machinecharts zichtbaar en niet leeg renderen
- chart-renderer unit tests controleren PNG-output voor belangrijke charttypes
- API tests controleren dashboard/capacity/value response-vormen
- route tests bewaken dat data payloads beschikbaar blijven

2. Handmatig:

- de human-verification workbook bevat expliciete chart- en heatmapstappen
- TEST C+D controleert onder andere utilization chart, heatmap thresholds, FTE chart en stacked OEE/demand/availability effecten

Nog niet volledig automatisch afgedekt:

- screenshot-regressie met baseline images
- automatische verificatie dat een specifieke chartlijn exact beweegt na een specifieke edit

Aanbevolen uitbreiding:

- na Line 09 edit controleren dat utilization chart data of pixels veranderen
- na Line 12 edit controleren dat FTE/value chart data of pixels veranderen

Waarom deze testset representatief is

De combinatie is representatief omdat elke laag een ander risico afdekt:

Testlaag	Dekt af
Unit tests	Formules en lokale engine-logica
Cascade tests	Interactieve edit-effecten en downstream herberekening
State tests	Reset, replay, persistence en session switch
Route tests	API-contracten en server-side workflows
Browser tests	Echte gebruikerflow door DOM + API + Flask
Human workbook	Cross-feature interacties, charts, heatmaps en visuele sanity
Golden-fixture tests	Numerieke vergelijking met een beheerde referentie-uitkomst
Pipeline regressies	Stabiliteit van belangrijke berekeningen
Performance guardrails	Grote pending-edit replay en grote synthetische resultsets

Samen geven deze tests vertrouwen dat:

- berekeningen numeriek consistent blijven
- edits downstream doorwerken
- sessies en restarts deterministisch blijven
- dashboards en exports data uit dezelfde engine state gebruiken
- handmatige gebruikersflows overeenkomen met de automatische checks

Bekende beperkingen

- De gemeten branch-enabled code coverage is 81%. Dat is sterk voor de huidige fase, maar nog geen volledige dekking.
- Chartcorrectheid is deels automatisch gevalideerd met nonblank canvas checks en deels handmatig via de human workbook. Pixel-perfect screenshot-regressies zijn nog niet ingericht.
- Performance heeft nu basis-guardrails voor grote replay/resultset-paden, maar zeer grote realistische workbooks, concurrent gebruik en uitzonderlijke infrastructuurfoutpaden zijn nog beperkt automatisch getest.
- Er is een bekende pandas-waarschuwing in `modules/cycle_manager.py` rond `select_dtypes(include=["object"])` ; dit is geen testfailure, maar wel een compatibiliteitspunt richting pandas 4.

Eindoordeel voor klantreview

De applicatie heeft een stevige automatische testbasis voor de belangrijkste reken- en gebruikersflows. De gemeten branch-enabled coverage-score is nu **81% voor `modules/` en `ui/` samen**. De meest kritieke klantpaden worden daarnaast gecontroleerd met 19 browsertests en gestructureerde handmatige validatie.

Voor productievertrouwen is het huidige testniveau bruikbaar voor een klantreview, mits de bekende beperkingen expliciet worden benoemd. De belangrijkste volgende stap is niet meer basisdekking, maar gerichte edit-to-chart-change checks, screenshot-regressie waar zinvol, en verdere performance- en foutpadvalidatie voor uitzonderlijke situaties.